

BCC1313
ALGORITHMS AND COMPLEXITY

Recursion Concepts

INTRODUCTION

- **Recursion** is a powerful computational tool that can significantly reduce the complexity of solving problems for programmers.
- It refers to a method where a **function calls itself**, either directly or indirectly, to address a given task.
- Recursion helps break problems down into smaller, **manageable sub-problems**, continuing the process until a **base case** is reached, making the problem easier to solve.

RECURSION vs ITERATION

- In programming, both **recursion** and **iteration** (**looping**) are essential techniques for solving problems.
- **Iteration** uses looping constructs such as `for`, `while`, or `do-while` to repeatedly execute a set of instructions until a condition is satisfied.

RECURSION vs ITERATION

- Example (using iteration):

  A program that displays the numbers from 10 down to 0:

Iteration

```
void countdown(int number) {
    for (int i = number; i >= 0; i--) {
        cout << i <<endl;
    }
}
```

RECURSION vs ITERATION

- Example (using recursion):

  A program that displays the numbers from 10 down to 0:

Recursion

```
void countdown(int number) {
    if (number < 0) {
        return; // Base case
    }
    cout<< number <<endl;
    countdown(number - 1); // Recursive call
}
```

RECURSION vs ITERATION

```
countdown(10);
```

- The walk-through (step-by-step) in recursion:
  - Step 1: Call countdown(10), so the argument number holds a 10.
  - Step 2: Print number (which contains the value 10) to the console.
  - Step 3: Before the countdown function is complete, it calls countdown(9).
  - Step 4: In countdown(9), begins running and print number (which is currently 9) to the console.
  - Step 5: Before countdown(9) is complete, it calls countdown(8).
  - Step 6: Countdown(8) begins running. Print 8 to the console.
  - ……..

RECURSION vs ITERATION

- In almost any case in which we can use iteration solution, we can also use recursion.
- Recursion is a tool that allows for writing elegant code.
- However, iteration offers more direct control and is typically easier to optimize for execution speed. Why?

THE BASE CASE

- Let’s continue our walkthrough of the countdown function:
  - …… skip a few steps
  - Step 21: Call countdown(0).
  - Step 22: Print number 0 to the console.
  - Step 23: Call countdown(-1).
  - Step 24: Print number (-1) to the console.
- This solution is not perfect, as we’ll end up stuck with infinitely printing **negative numbers**. How to find a way to prevent the recursion from continuing on forever?

THE BASE CASE

- To solve this problem, add the conditional statement:

```
void countdown(int number) {
    cout<<number<<endl;

    if (number == 0) { // Base case
        return;
    } else {
        countdown(number - 1); // Recursive call
    }
}
```

- In this case in which the method will **not recurse** is known as the **base case**.

RECURSION APPLICATIONS

- The concept of recursion originated in the realm of mathematics.
- It was found that some interesting mathematical sequences could be defined in terms of themselves, which greatly simplified their definitions.
- Example 1: **Fibonacci sequence**

{0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, … }

FIBONACCI SEQUENCE

- The sequence starts at 0 and followed by 1, and each subsequent value in the sequence is derived from the previous two elements in the sequence.

$F_0 = 0$

$F_1 = 1$

$F_2 = 1$

$F_3 = 2$

$F_4 = 3.$

$F_n = ?$

Fibonacci formula:

$F_n = F_{n-1} + F_{n-2}$

FIBONACCI SEQUENCE

- Let say, to find the eight sequences:

$$F_8 = F_{8-1} + F_{8-2}$$

$$= F_7 + F_6$$

$$= (F_{7-1} + F_{7-2}) + (F_{6-1} + F_{6-2})$$

FIBONACCI SEQUENCE

- The algorithms for Fibonacci sequence:
  - Base case:

n = 0 the algorithm should return 0
n = 1 the algorithm should return 1

  - Recursive case:

any other n the algorithm returns the sum of Fibonacci(n − 1) and Fibonacci(n − 2)

FIBONACCI SEQUENCE

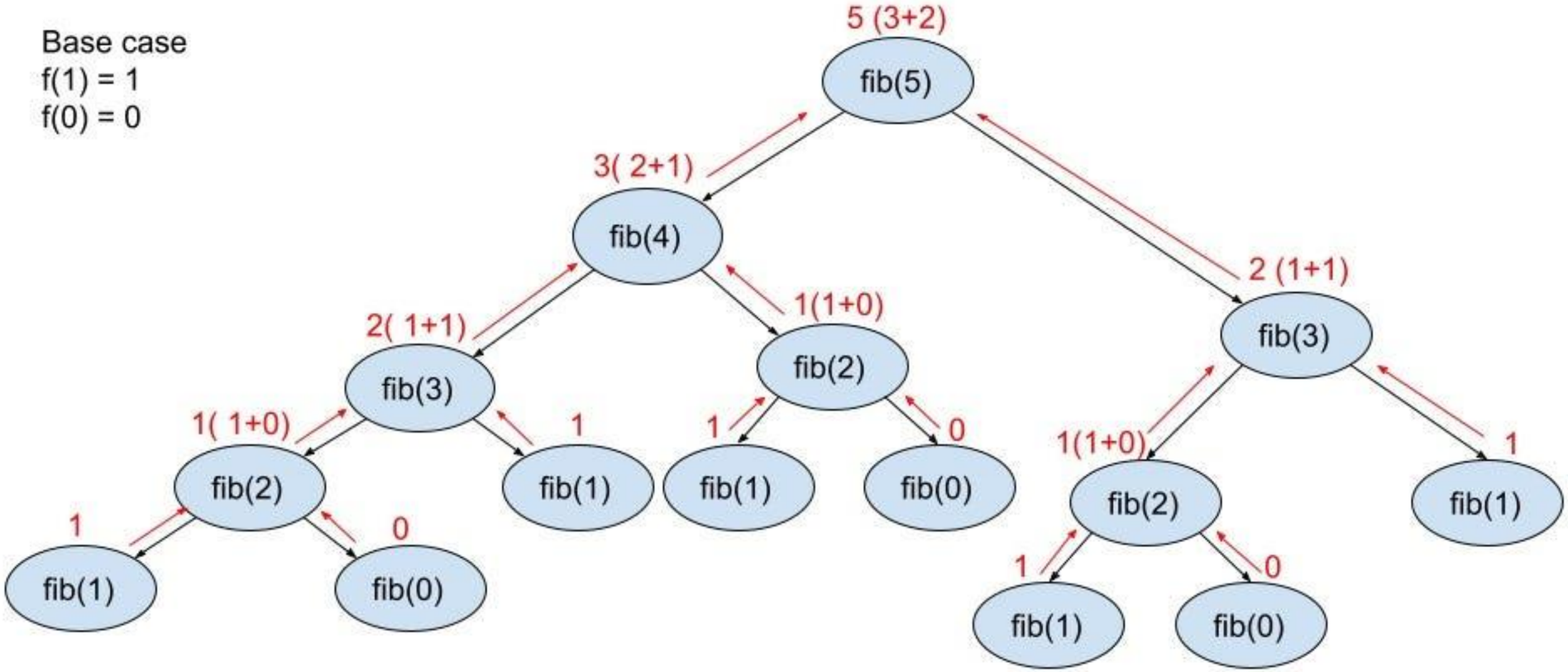
- The solution:

```
int main() {
    int n = 5;
    for (int i = 0; i <= n; ++i) {
        cout << fibonacci(i) << " ";
    }
    return 0;
}
```

```
int fibonacci(int n) {
    if (n <= 1)
        return n; // Base cases: F(0) = 0, F(1) = 1
    return fibonacci(n - 1) + fibonacci(n - 2); // Recursive calculation
}
```

FIBONACCI SEQUENCE

- Drawing solution for F(5):



FIBONACCI SEQUENCE

- Practice:
  - Draw the solution for F(8)

RECURSION APPLICATIONS

- Example 2: **Factorial numbers**
- A factorial of a non-negative integer n is the product of all positive integers less than or equal to n. It's denoted by n! and is defined as:

$$n! = n \times (n-1) \times (n-2) \times \cdots \times 1$$

$$5! = 5 \times 4 \times 3 \times 2 \times 1 = 120$$

$$8! = ???$$

FACTORIAL NUMBERS

- Factorials are often defined recursively where:
  - Base case:

    If `n == 0`, the function returns 1 $0! = 1$
  - Recursive case:

    If `n` is greater than `0`, the function calls itself with `n - 1`

    and multiplies `n` by the result of that call.

$$n! = n \times (n-1)!$$

FACTORIAL NUMBERS

- Iterative factorial algorithm using a loop:

```
Algorithm iterativefactorial (val n <integer>)

1 i = 1
2 factN = 1
3 loop (i <= n)
   1 factN = factN * i
   2 i = i + 1
4 end loop
5 return factN

End iterativeFactorial
```

FACTORIAL NUMBERS

- Recursive factorial algorithm:

```
Algorithm recursiveFactorial (val n <integer>)

1 If (n equal 0)
   1 Return 1
2 Else
   1 Return (n * recursiveFactorial(n - 1))
3 End if

End factorial
```

FACTORIAL NUMBERS

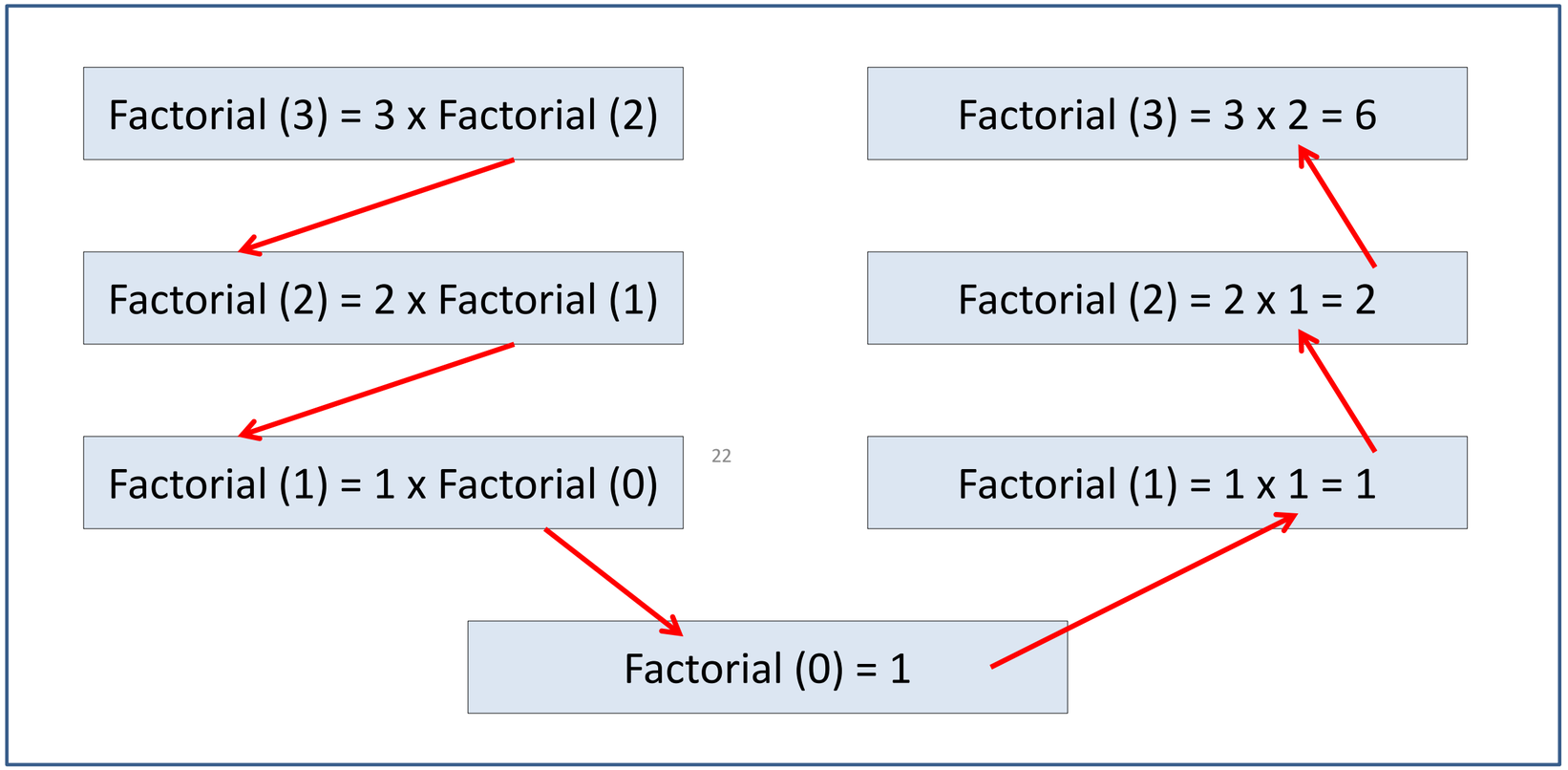
- The solution:

```
int main() {
    int number = 5;
    cout<<"Factorial of "<<number<<" is: "<<factorial(number)<<endl;
    return 0;
}
```

```
int factorial(int n) {
    if (n == 0) // Base case: 0! = 1
        return 1;
    else
        return n * factorial(n - 1); // Recursive case: n! = n * (n - 1)!
}
```

FACTORIAL NUMBERS

- Drawing solution for factorial(3):



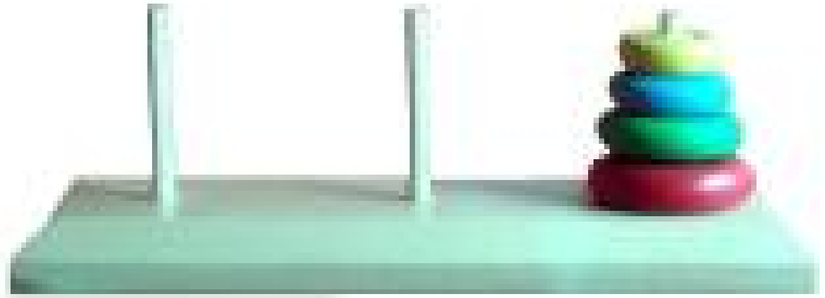
FACTORIAL NUMBERS

- Practice:
  - Draw the solution for factorial(7)

RECURSION APPLICATIONS

- Example 3: **Tower of Hanoi**
- It is a classic problem that involves three rods and a set of disks of varying sizes, which can be placed on any rod. The goal is to transfer all the disks from the first rod to the third rod, adhering to the following rules:
  - Only one disk can be moved at a time.
  - A disk can only be moved to the top of another rod.
  - A disk cannot be placed on top of a smaller disk.

TOWER OF HANOI



- Base Case:

  If there is only one disk, it is simply moved from the source to the destination.

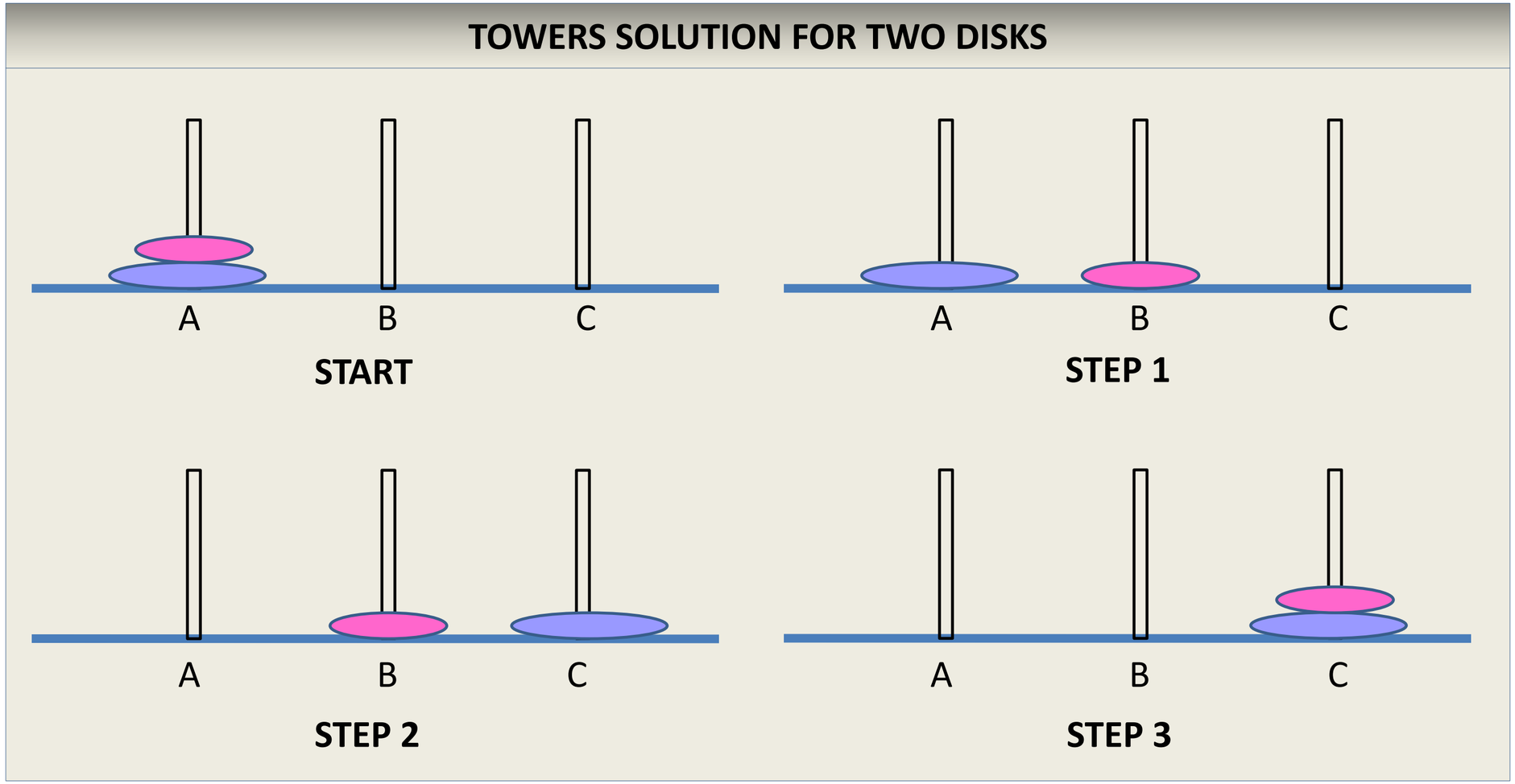
- Recursive Case:

  If there are more than one disk:

  1. The first n-1 disks are moved from the source rod to the auxiliary rod.
  2. The largest disk is moved from the source rod to the destination rod.
  3. The n-1 disks are moved from the auxiliary rod to the destination rod.

TOWER OF HANOI

A = Source
B = Auxiliary
C = Destination



TOWER OF HANOI

- The solution:

```
int main() {
    int n = 3; // Number of disks
    towerOfHanoi(n, 'A', 'C', 'B');
    return 0;
}
```

```
void towerOfHanoi(int n, char source, char destination, char auxiliary) {
    if (n == 1) { // Base case: Only one disk to move
        cout<<"Move disk 1 from "<<source<<" to "<<destination<<endl;
        return;
    }

    // Step 1: Move n-1 disks from source to auxiliary
    towerOfHanoi(n - 1, source, auxiliary, destination);

    // Step 2: Move nth disk from source to destination
    cout<<"Move disk "<<n<<" from "<<source<<" to "<<destination<<endl;

    // Step 3: Move n-1 disks from auxiliary to destination
    towerOfHanoi(n - 1, auxiliary, destination, source);
}
```

SUMMARY

- Recursive solution does not need a loop, recursion is itself repetition.
- Recursion let the algorithm to call itself, each time with a different set of parameters.

PRACTICE

1. What would be returned if fun1 is called as…
    a) fun1(4)
    b) fun1(10)
    c) fun1(12)

```
int fun1(int x) {
    if (x < 5) {
        return 3 * x;
    } else {
        return 2 * fun1(x - 5) + 7;
    }
}
```

PRACTICE

2. What would be returned if fun2 is called as…

   a) fun2(2,7)

   b) fun2(5,3)

   c) fun2(15,3)

```
int fun2(int x, int y) {
    if (x < y) {
        return -3;
    } else {
        return fun2(x - y, y + 3) + y;
    }
}
```

PRACTICE

3. What would be returned if fun3 is called as…

   a) fun3(10,4)

   b) fun3(4,3)

   c) fun3(4,7)

   d) fun3(0,0)

```
int fun3(int x, int y) {
    if (x > y) {
        return -1;
    } else if (x == y) {
        return 1;
    } else {
        return x * fun3(x + 1, y);
    }
}
```

PRACTICE

- Tutorial 2
- Lab 2